

MedAssist AI — Comprehensive Project Architecture & Specification

End-to-End Technical Documentation from Scratch | Clinical Diagnostic Platform

1. Project Overview & Clinical Mission

MedAssist AI is a full-stack, AI-powered healthcare intelligence platform designed to assist both patients and healthcare providers in preliminary symptom analysis, machine learning-driven disease prediction, clinical risk stratification, treatment recommendations, and automated medical report generation.

Key Problems Solved:

- **Diagnostic Guidance:** Provides rapid, evidence-backed preliminary disease probability assessments based on patient symptom profiles.
- **Risk Stratification:** Categorizes clinical cases into Low, Moderate, High, or Critical urgency tiers based on symptom duration and severity.
- **Actionable Treatment Plans:** Offers structured medical recommendations, medications, diet guidelines, and lifestyle precautions.
- **Automated Clinical Reporting:** Generates downloadable, publication-grade PDF medical reports complete with patient history and diagnostic summaries.
- **Seamless Authentication:** Provides multi-channel user authentication featuring JWT, OAuth 2.0 (Google & Microsoft), and interactive fallback capabilities.

2. System Architecture & Technology Stack

Layer	Technology / Framework	Version / Tools	Description & Purpose
Frontend UI	React.js	v18.3.1	Single Page Application (SPA) with responsive dark-glass UI
Build Tool	Vite	v5.4.21	Lightning-fast frontend bundler & dev server
Styling & UX	Tailwind CSS	v3.4.17	Utility-first styling with custom glassmorphism design tokens
Icons & Motion	Lucide React & Framer Motion	Latest	Modern iconography, smooth page transitions & interactive modals
Backend API	FastAPI (Python)	Python 3.14 / FastAPI 0.115+	Asynchronous REST API framework with automatic OpenAPI docs
Database & ORM	SQLAlchemy & SQLite	SQLAlchemy v2.0+	Relational database schema with repository pattern data access
AI / ML Models	Scikit-Learn & NumPy	v1.5+	Random Forest & Decision Tree classification models for diagnostic prediction
Security & Auth	PyJWT & Passlib	v2.9+	JSON Web Token (JWT) stateless auth with bcrypt password hashing
PDF Generator	ReportLab	v5.0.0	Automated canvas & document generation for medical reports

3. Complete Project Directory Structure

The project is structured cleanly as a decoupled monorepo containing distinct **frontend** and **backend** services:

```
medassist-ai/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── deps.py          # API dependencies (database session, current user JWT validation)
│   │   │   ├── routers/
│   │   │   │   ├── auth.py      # Login, Register, Google & Microsoft OAuth endpoints
│   │   │   │   ├── dashboard.py # Health stats & analytical summary endpoints
│   │   │   │   ├── predictions.py # Disease prediction submission & history API
│   │   │   │   ├── recommendations.py # Medication & lifestyle recommendation API
│   │   │   │   ├── reports.py   # PDF medical report generation & download API
│   │   │   │   ├── risk.py      # Clinical urgency & risk assessment API
│   │   │   │   ├── symptoms.py  # Symptom search & catalog endpoints
│   │   │   │   └── users.py     # User profile management API
│   │   ├── core/
│   │   │   ├── config.py        # Application settings & environment variables
│   │   │   ├── database.py      # SQLAlchemy engine & session factory
│   │   │   ├── security.py      # JWT token encoding/decoding & bcrypt hashing
│   │   │   └── ml/
│   │   │       ├── predictor.py  # Machine learning model loader & inference engine
│   │   │       ├── recommendation_engine.py # Clinical treatment & medication mapper
│   │   │       ├── risk_engine.py # Risk score calculator & severity classifier
│   │   │       └── models/      # SQLAlchemy ORM Data Models (User, Patient, Symptom, etc.)
│   │   └── tests/
│   └── scripts/
```

```

■ ■ ■■ repositories/          # Data Access Layer (UserRepository, PredictionRepository, etc.)
)
■ ■ ■■ schemas/              # Pydantic Request/Response validation schemas
■ ■ ■■ services/             # Business logic & ReportLab PDF generator
■ ■ ■■ main.py               # FastAPI application entry point & CORS configuration
■ ■■ migrate_db.py          # Automatic database migration & schema updater
■ ■■ requirements.txt        # Python dependencies specification
■ ■■ sql_app.db              # SQLite database instance
■■■ frontend/
■ ■■ src/
■ ■ ■■ components/           # UI Reusable Primitives (Navbar, Sidebar, RippleButton, etc.)
■ ■ ■■ context/              # React Contexts (AuthContext.jsx, ThemeContext.jsx)
■ ■ ■■ hooks/                 # Custom Hooks (usePrediction.js, useAuth.js)
■ ■ ■■ pages/
■ ■ ■ ■■ Auth/               # LoginPage, RegisterPage, Google & Microsoft Modals
■ ■ ■ ■■ Dashboard/          # Dashboard Analytics & Quick Actions
■ ■ ■ ■■ DiseasePrediction/   # Interactive AI Diagnosis Form & Results
■ ■ ■ ■■ Landing/             # Marketing Landing Page & Features Hero
■ ■ ■ ■■ Profile/             # User Profile Settings & Medical History
■ ■ ■ ■■ Recommendations/     # Treatment Plans & Prescriptions View
■ ■ ■ ■■ Reports/             # Downloadable Clinical PDF Reports Center
■ ■ ■ ■■ RiskAssessment/      # Triage & Urgency Assessment View
■ ■ ■ ■■ SymptomAnalysis/     # Multi-symptom selector & severity evaluator
■ ■ ■■ routes/               # AppRoutes.jsx, ProtectedRoute.jsx, PublicOnlyRoute.jsx
■ ■ ■■ services/             # Axios API clients (authService.js, apiService.js)
■ ■ ■■ App.jsx               # Application root component
■ ■ ■■ main.jsx              # Vite React DOM entry point
■ ■■ package.json            # Node.js dependencies & scripts
■ ■■ vite.config.js          # Vite build configuration
■■■ README.md                # Project documentation overview

```

4. Key Modules & Functional Workflows

A. Authentication & Security Flow

- **Form-First Layout:** The login portal features an intuitive form layout placing Email & Password inputs at the top, followed by the main 'Sign In to Portal' action button, and Social Auth buttons (Google & Microsoft) below.
- **Dual OAuth & Dev Fallback:** Supports standard Google/Microsoft OAuth 2.0 logins. In development environments where OAuth Client IDs are pending, integrated fallback account selector modals allow instant testing.
- **Token Persistence:** Valid JWT tokens are stored securely in ``localStorage`` and attached via Axios request interceptors to all protected API calls.

B. AI Symptom Analysis & Disease Prediction

- **Symptom Search & Selection:** Patients select symptoms from a curated clinical database, specifying severity levels (Mild, Moderate, Severe) and duration (in days).
- **Machine Learning Inference:** The backend ML engine (``predictor.py``) processes input vectors through pre-trained classification models to predict target medical conditions with calculated confidence scores.
- **Full Flow Integration:** Submitting symptom analysis automatically creates linked Database records for Prediction, Risk Assessment, and Recommendations.

C. Clinical Risk Assessment & Triage Engine

- **Urgency Stratification:** Analyzes symptom severity and duration to output a clinical risk tier: *Low (Green)*, *Moderate (Yellow)*, *High (Orange)*, or *Critical (Red)*.
- **Dynamic Guidance:** Recommends specific timeframes for medical consultation (e.g., immediate ER visit vs routine doctor appointment within 48 hours).

D. Treatment & Recommendation Engine

- **Medication Guidance:** Provides over-the-counter and prescription medicine suggestions along with dosage guidelines, indications, and precautions.
- **Holistic Care Plans:** Delivers dietary guidelines, fluid intake recommendations, physical activity advice, and specialist doctor referrals.

E. ReportLab PDF Medical Report Generator

- **Publication-Grade PDF:** Built with ReportLab, generating standard formatted PDF medical reports containing patient demographics, diagnostic predictions, risk breakdown, and prescriptions.
- **Direct Browser Download:** Accessible directly from the Reports page via backend ``/api/v1/reports/download/{id}`` endpoint.

5. Relational Database Schema Specification

Table Name	Primary Key	Key Fields	Foreign Keys & Relations
users	id (Integer)	email, hashed_password, full_name, role, is_active	One-to-One with patient, One-to-Many with notifications
patients	id (Integer)	age, gender, blood_type, height, weight, allergies	FK -> users.id, One-to-Many with predictions
symptoms	id (Integer)	code, name, body_part, severity, synonyms	Catalog lookup table for ML input matching
diseases	id (Integer)	code, name, category, description, specialist	Catalog table for predicted health conditions
predictions	id (Integer)	primary_disease, confidence, symptoms_list	FK -> patients.id, Has-One Risk & Recommendation
risk_assessments	id (Integer)	risk_level, urgency, risk_factors	FK -> predictions.id
recommendations	id (Integer)	medicines, lifestyle, diet, follow_up	FK -> predictions.id
reports	id (Integer)	report_code, pdf_path, generated_at	FK -> predictions.id, FK -> patients.id

6. Implementation Status Matrix

Feature Area	Implementation Status	Details & Capabilities
Email & Password Auth	100% Fully Implemented	BCrypt password hashing, JWT access token issue & validation
Google & Microsoft Sign-In	100% Fully Implemented	OAuth 2.0 flow with interactive modal fallbacks for development
Symptom Analysis	100% Fully Implemented	Interactive symptom tag selection, severity and duration input
AI Disease Prediction	100% Fully Implemented	Machine Learning classification with confidence score outputs
Risk Stratification	100% Fully Implemented	Urgency score, risk matrix calculation, and emergency flags
Treatment & Prescriptions	100% Fully Implemented	Medication recommendations, lifestyle, diet, and specialist advice
PDF Report Generation	100% Fully Implemented	ReportLab engine PDF export and browser download integration
User Dashboard	100% Fully Implemented	Recent predictions summary, health status stats, and quick actions

7. Setup & Execution Guide (From Scratch)

Follow these exact steps to run the backend and frontend locally from scratch:

```
# 1. Backend Setup & Run
cd backend
python -m venv venv
# On Windows:
venv\Scripts\activate
pip install -r requirements.txt
```

```
python migrate_db.py
uvicorn app.main:app --reload --port 8000

# 2. Frontend Setup & Run (in a separate terminal)
cd frontend
npm install
npm run dev

# Access the Web Application at:
http://localhost:5173
# API Documentation (Swagger UI) at:
http://localhost:8000/docs
```